

Informatik 2

Kapitel 5 – Java-API

Inhaltsverzeichnis

5.1	Einleitung	3
5.2	Strings	3
5.2.1	Schnittstellen für Strings	3
5.2.2	Die Klasse String	4
5.2.3	Die Klasse StringBuilder	5
5.3	Unveränderliche Klassen	6
5.3.1	Hüllklassen	8
5.3.2	Datum und Zeit	10
5.3.3	Fallstudie grafischer Editor: Eine unveränderliche Klasse Point	12
5.4	Parametrisierte Klassen und Schnittstellen	13
5.4.1	Sammlungen verwalten (ArrayList<T>)	13
5.4.2	Parametrisierte Klassen	15
5.4.3	Sammlungen durchlaufen (Iterable<T>)	15
5.4.4	Objekte vergleichen (Comparable<T>)	17
5.4.5	Parametrisierte Schnittstellen	18
5.4.6	Weiterführende Konzepte	19
5.5	Collection-API	20
5.5.1	Grundlagen	20
5.5.2	Fallstudie grafischer Editor: Container-Klassen	23
5.5.3	Die forEach - Methode	26
5.6	Lambda-Ausdrücke	30
5.6.1	Funktionale Schnittstellen	30
5.6.2	Methoden- und Konstruktorreferenzen	34

5.1 Einleitung

In diesem Kapitel lernen wir einige allgemein nützliche Datentypen der Java-API (Klassen und Schnittstellen) kennen, die man zur Verwaltung von Daten größerer Anwendungen immer wieder benötigt. Dazu gehören Typen für

- **Strings**
- **Zahlen und Zeichen**
- **Datums- und Zeitangaben**
- **Sammlungen von Objekten (Collections)**
- **Datenströme (Streams)**

Einige wichtige Typen sind dabei **parametrisiert** (solche Typen heißen **Generics**), was sie noch flexibler einsetzbar macht.

Zudem lernen wir auch erste wichtige Techniken und Konzepte für die Implementierung einer gut wartbaren, wenig fehleranfälligen und möglichst allgemein einsetzbaren Software kennen, die wir mit Hilfe der API umsetzen können. Dazu zählen **funktionale Schnittstellen** und **Lambda-Ausdrücke**.

5.2 Strings

Anders als in C sind Strings in Java **keine** Arrays, sondern ein eigener Datentyp. In Java gibt es drei verschiedene Klassen inklusive Verwaltungsoperationen zur Repräsentation von Strings:

- `String`
- `StringBuffer`
- `StringBuilder`

Bis auf die Klasse `StringBuffer` wollen wir diese Klassen in diesem Kapitel kurz näher betrachten.

5.2.1 Schnittstellen für Strings

Der Zugriff auf einzelne Zeichen eines Strings kann in jedem Fall nur **über geeignete Verwaltungsmethoden** erfolgen. All erwähnten Klassen implementieren dazu unter anderem die Schnittstelle `CharSequence`, womit sie etwa die folgenden wichtigen (gemeinsamen) Methoden zur Verfügung stellen:

- `char charAt(int index):`

Liefert das Zeichen an Position `index` eines Strings.

Beispielaufruf für ein String-Objekt `s`:

```
s.charAt(0); // liefert das erste Zeichen von s
```

- `int length()`:
Liefert die Länge eines Strings.
Beispielaufruf für ein String-Objekt `s`:
`s.length()`;

Davon abgesehen implementieren diese Klassen auch die Schnittstelle `Comparable<T>`. Dabei handelt es sich um eine sogenannte **parametrisierte Schnittstelle**. Das ist eine Schnittstelle, die Methoden anbietet, die auf die Klasse angepasst sind, die man für den Parameter `T` einsetzt (Details dazu folgen im Unterkapitel 5.5):

- Konkret implementiert die Klasse `String` die Schnittstelle `Comparable<String>` und damit die Methode

```
int compareTo( String another )
```

Für `T` wird also `String` eingesetzt.

- Die Klasse `StringBuffer` implementiert entsprechend die Methode

```
int compareTo( StringBuffer another )
```

Für `T` wird in diesem Fall angepasst `StringBuffer` eingesetzt.

- Analog implementiert die Klasse `StringBuilder` die Methode

```
int compareTo( StringBuilder another )
```

und für `T` wird `StringBuilder` eingesetzt.

Bei diesen `compareTo`-Implementierungen werden Strings jeweils bezüglich der lexikographischen Ordnung verglichen.

5.2.2 Die Klasse `String`

Wir haben die Klasse `String` schon in vielen Beispielen kennengelernt und diskutiert. Ausgewählte `String`-Konstruktoren sind:

- `String()`:
Leerer Konstruktor: Erzeugt leeren String
- `String(String original)`:
Kopierkonstruktor: Erzeugt String mit demselben Inhalt wie `original`.
- `String(StringBuilder builder)`:
Erzeugt String mit demselben Inhalt wie `builder`.

Einige ausgewählte Methoden der Klasse `String` sind:

- `String toUpperCase()`:
Erzeugt einen neuen String, in dem alle Buchstaben in Großbuchstaben umgewandelt wurden, und gibt diesen zurück

- `boolean startsWith(String prefix):`
Überprüft, ob `prefix` ein Anfangsstück des Strings ist
- `String concat(String str):`
Erzeugt einen neuen String durch Anhängen von `str` und gibt diesen zurück
- `String replace(char oldChar, char newChar):`
Erzeugt einen neuen String, in der alle Vorkommen von `oldChar` durch `newChar` ersetzt wurden, und gibt diesen zurück
- `static String valueOf(<Typ> val):`
Gibt `val` umgewandelt in `String` zurück (für verschiedene Datentypen `<Typ>`, überladene Methode)

Objektmethoden der Klasse `String` verändern also nie den Inhalt des Objekts, sondern erstellen eine Kopie mit verändertem Inhalt und geben diese zurück. Solche Klassen nennt man **unveränderlich (immutable)**, da der Inhalt von Objekten solcher Klassen unveränderlich ist. Wir fassen hier kurz zusammen:

Definition 5.1: Klasse `String`

Die Klasse `String` repräsentiert **unveränderliche Strings**, deren Inhalt nicht geändert werden kann. Es gibt deshalb keine Methoden zum Ändern des Inhalts eines einmal konstruierten `String`-Objekts.

- Methoden führen Modifikationen am Inhalt nur auf einer Kopie aus und liefern eine Referenz auf die Kopie. Diese Referenz kann mit

```
s = <Methodenaufruf>;
```

einem `String`-Objekt `s` zugewiesen werden, womit man Zugriff auf den modifizierten Inhalt erhält.

- `String`-Objekte können mit dem binären Operator `+` aneinandergehängt werden.
- In Apostrophe eingeschlossen Zeichenfolgen der Form

```
"<Zeichenfolge>"
```

sind **Konstanten vom Typ `String`**.

5.2.3 Die Klasse `StringBuilder`

Die Klasse `StringBuilder` repräsentiert als Alternative zu `String` Strings, deren Inhalt veränderlich ist. Deshalb hat jedes `StringBuilder`-Objekt eine Kapazität, die bestimmt, wie viel Speicherplatz für das Objekt zur Verfügung gestellt wird. Diese Kapazität ist immer automatisch an die Länge des verwalteten Strings angepasst. Ausgewählte Konstruktoren sind:

- `StringBuilder():`
Leerer Konstruktor: Erzeugt leeren String mit voreingestellter Anfangskapazität (16 Zeichen).
- `StringBuilder(int capacity):`
Erzeugt leeren String mit Anfangskapazität `capacity`.

- `StringBuilder(String value)`:
Erzeugt String mit Inhalt `value` und daran angepasster Kapazität.

Ausgewählte Methoden der Klasse `StringBuilder` sind:

- `StringBuilder insert(int offset, String str)`:
Fügt `str` an der Position `offset` ein.
- `StringBuilder insert(int offset, int i)`:
Fügt `String.valueOf(i)` an der Position `offset` ein.
- `StringBuilder delete(int start, int end)`:
Löscht die Zeichen zwischen Position `start` und Position `end-1`.
- `int capacity()`:
Liefert die aktuelle Kapazität.

Die `insert`-Methode ist ein weiteres Beispiel für eine überladene Methode.

Definition 5.2: Klasse `StringBuilder`

Die Klasse `StringBuilder` repräsentiert **veränderliche Strings**, deren Inhalt beliebig verändert werden kann. Die (Speicher-)**Kapazität** zur Aufnahme des Strings richtet sich nach dessen Länge und wird automatisch angepasst. Methoden führen Modifikationen direkt auf dem Inhalt eines `StringBuilder`-Objekts `s` aus. Ein Aufruf

```
s.<Methodenaufruf>;
```

verändert den Inhalt von `s`. Eine Wertzuweisung ist also nicht notwendig. Andererseits ist kein Zugriff auf den alten Inhalt mehr möglich.

5.3 Unveränderliche Klassen

Nach der Klasse `String` wollen wir uns nun weitere unveränderliche Klassen ansehen und deren Eigenschaften allgemein definieren und genauer untersuchen.

Definition 5.3: Unveränderliche Klassen

Eine Klasse heißt **unveränderlich**, wenn der Zustand eines Objekts der Klasse (also dessen Attributwerte) nach dessen Konstruktion nicht mehr verändert werden kann – auch nicht indirekt über Referenzen.

Eine unveränderliche Klasse hat deshalb folgende **Kerneigenschaften**:

1. Alle Attribute sind `final`. Dadurch können nur einmal gesetzt werden.
2. Alle Attribute sind `privat`. Dadurch ist kein direkter Zugriff von außen möglich.
3. Es gibt keine Set-Methoden (diese wären nutzlos, da die Attribute `final` sind).

4. Die Klasse ist final. Dadurch können keine Unterklassen mit anderem Verhalten gebildet werden.

Unveränderliche Klassen haben unter anderem folgende Vorteile:

- Ihre Benutzung ist weniger fehleranfällig, da keine versteckten Zustandsänderungen über Referenzen möglich sind.
- Ihre Benutzung ist thread-sicher (siehe Kapitel 14).

Insbesondere kann man unveränderliche Klassen als Datentypen von Attributen wie primitive Datentypen verwenden. Zum Beispiel können wir mit der Get-Methode `getName` einer Klasse `Person` direkt wie folgt die Original-Referenz des `String`-Attributs `name` zurückgeben (genauso wie bei primitiven Datentypen):

```
public String getName() {  
    return this.name;  
}
```

Rufen wir in einer anderen Klasse diese Get-Methode wie folgt für ein `Person`-Objekt `p` auf:

```
String s = p.getName();
```

so können wir über `s` den Namen von `p` nicht ändern, da die Klasse `String` unveränderlich ist.

Dasselbe sollten wir aber nicht mit der Get-Methode `getPosition` der Klasse `Form` machen:

```
public Point getPosition() {  
    return this.position;  
}
```

Rufen wir hier in einer anderen Klasse diese Get-Methode wie folgt für ein `Form`-Objekt `f` auf:

```
Point p = f.getPosition();
```

so können wir über `p` die Position von `f` ändern, da die Klasse `Point` veränderlich ist, etwa durch:

```
p.setX(10);
```

Eine solche Änderung der Position sollte allerdings nach dem Einkapselungsprinzip nur über die `setPosition`-Methode von `Form` möglich sein. Deshalb hatten wir dafür die folgende Implementierung verwendet, in der nur eine Kopie zurückgegeben wird:

```
public Point getPosition() {  
    return new Point(this.position);  
}
```

Wir sehen uns nun einige unveränderliche Klassen der API an. Die Klassen können wir also wie primitive Datentypen für Attribute eigener Datenklassen verwenden. Anschließend entwerfen wir exemplarisch eine neue unveränderliche Version unserer Klasse `Point`.

5.3.1 Hüllklassen

Gelegentlich kann es nützlich sein, einen primitiven Datentyp wie ein Objekt zu behandeln. Zum Beispiel erlaubt die Bibliotheksklasse `ArrayList` nur die Verwaltung von Objekten, sodass wir in einer solchen zum Beispiel keine `int`- oder `double`-Werte speichern können. Um dieses Problem aus der Welt zu schaffen, stellt Java für alle primitiven Datentypen sogenannte **Hüllklassen** (**wrapper classes**) zur Verfügung:

Hüllklasse	primitiver Datentyp
<code>Boolean</code>	<code>boolean</code>
<code>Character</code>	<code>char</code>
<code>Number</code>	alle Zahlklassen
<code>Integer</code>	<code>int</code>
<code>Long</code>	<code>long</code>
<code>Float</code>	<code>float</code>
<code>Double</code>	<code>double</code>

Diese Hüllklassen sind ebenso wie `String` unveränderlich und *verpacken* die primitiven Datenwerte in Objekte. Sie ermöglichen so die Anwendung verschiedener Methoden, die von den Hüllklassen zur Verfügung gestellt werden.

Wollen wir den Wert eines primitiven Datentyps zu einem Objekt machen, so haben wir dazu mehrere Möglichkeiten. Eine davon ist die Nutzung des Konstruktors der entsprechenden Hüllklasse, doch das sollten wir **unbedingt vermeiden**, da dieser in der API als **veraltet (deprecated)** markiert ist. Das bedeutet, er wird in einer zukünftigen Java-Version entfernt. Wenn wir also möchten, dass unsere Java-Programme auch in Zukunft noch laufen (und das wollen wir wohl so gut wie immer), sollten wir andere Möglichkeiten als einen Konstruktor verwenden.

Eine weitere Möglichkeit ist die Verwendung der Klassenmethode

```
public static <Huellklasse> valueOf(<Typ> value)
```

Diese liefert uns ein Objekt vom Typ der gewünschten Hüllklasse `<Huellklasse>`, das den Wert verwaltet, den wir mit `value` an die Methode übergeben haben. Da es sich um eine Klassenmethode handelt, müssen wir nur den Klassennamen kennen, um sie aufzurufen. Ein Aufruf mit Wertzuweisung sieht wie folgt aus:

```
<Huellklasse> o = <Huellklasse>.valueOf(<Wert>);
```

Dadurch wird ein Hüllobjekt `o` angelegt und erzeugt, das den Wert `<Wert>` verwaltet. Betrachten wir ein konkretes Beispiel:

```
Integer o = Integer.valueOf(5);
```

Hier wird der `int`-Wert 5 in ein Objekt vom Typ `Integer` verpackt. Als dritte Möglichkeit schreibt man einfach:

```
<Huellklasse> o = <Wert>;
```

Hier wird der primitive Wert automatisch verpackt. Das nennt man **Autoboxing** und ist die gebräuchlichste, weil kürzeste Form. Im Beispiel sieht das so aus:

```
Integer o = 5;
```

Umgekehrt lässt sich natürlich auch ein Wert, der in einem Hüllobjekt gespeichert wurde, wieder auslesen. Dies ist zuerst einmal mit der Objektmethode

```
public <Typ> <Typ>Value()
```

möglich. Wie bei allen Objektmethoden benötigt diese ein konkretes Objekt, auf das sie angewandt werden soll – ansonsten wäre nicht klar, was für ein Wert zurückgegeben werden soll. Haben wir ein konkretes Hüllobjekt `h`, so können wir mittels

```
<Typ> x = h.<Typ>Value();
```

in `x` den Wert abspeichern, der im Hüllobjekt `h` verwaltet wird. Konkret können wir also den Wert 5 aus unserem `Integer`-Objekt `o` des vorherigen Beispiels wieder gewinnen, indem wir die Anweisung

```
int x = o.intValue();
```

schreiben. Auch das geht noch einfacher mit:

```
<Typ> x = <Huellobjekt>;
```

Hier wird der Inhalt des Hüllobjekts automatisch entpackt. Das nennt man **Unboxing** und ist die gebräuchlichste, weil kürzeste Form. Im Beispiel sieht das so aus:

```
int x = o;
```

Man kann Hüllobjekte deshalb mit den Vergleichsoperatoren `<`, `<=`, `>` und `>=` direkt vergleichen. Dabei werden automatisch die Inhalte entpackt und verglichen.

Aber Achtung: Dies gilt nicht für Vergleiche mit `==` und `!=`, da hier die Referenzen verglichen werden und nicht die Inhalte!

Weitere Erklärungen und Beispiele zum Auto- und Unboxing finden Sie in der Java-Dokumentation: <https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html>

Hüllklassen stellen einige besondere Funktionalitäten bereit:

- **Konstanten für Grenzen des Wertebereichs:**

Die Hüllklassen stellen öffentliche konstante Klassenattribute zur Abfrage der Grenzen der Wertebereiche der primitiven Datentypen zur Verfügung, beispielsweise:

- Minimaler `int`-Wert: `Integer.MIN_VALUE`.
- Maximaler `char`-Wert: `Character.MAX_VALUE`.

- **Klasse `Character`:**

Die Klasse `Character` stellt (ähnlich wie die Bibliothek `ctype.h` in C) viele nützliche Methoden zur Verwaltung einzelner Zeichen zur Verfügung:

- Testmethoden: `isDigit`, `isLetter`, `isWhiteSpace`, `isLowerCase`, ...

- Umwandlungsmethoden: `toUpperCase`, ...
- **Umwandlung von Strings:**
 Die Hüllklassen (mit Ausnahme von `Character`) erlauben mit der statischen Methode `parse` `<Typ>` die Umwandlung eines Strings `s` nach `<Typ>` (ähnlich wie die `atoi`-Funktion in C).
 Beispiel:
 - Klasse `Integer`:

```
public static int parseInt(String s)
```

Beispielaufruf:

```
int x = Integer.parseInt("10");
```
 - Klasse `Double`:

```
public static double parseDouble(String s)
```

Beispielaufruf:

```
double x = Double.parseDouble("0.5");
```

5.3.2 Datum und Zeit

In vielen praktischen Anwendungen ist es notwendig, Datums- und Zeitangaben zu verarbeiten. Personenbezogene Daten benötigen zum Beispiel die Verwaltung des Geburtsdatums (siehe zum Beispiel die Klasse `Person` in Kapitel 4) und die Universität verwaltet unter anderem Prüfungstermine. Auch Zeitdauern spielen eine wichtige Rolle. So möchte man zum Beispiel messen, wie viele Nanosekunden ein Programm zur Verarbeitung bestimmter Daten benötigt.

Java verfügt über umfassende Möglichkeiten, um ein Datum oder eine Uhrzeit und auch Zeitdauern zu verarbeiten. Einige der dafür zur Verfügung stehenden Klassen und Methoden haben Sie schon in einzelnen Beispielen in den bisherigen Kapiteln (und den zugehörigen Lehrvideos) kennengelernt. Dabei haben Sie eventuell bereits bemerkt, dass sich diese Klassen etwas anders verhalten als die meisten anderen Klassen, wie zum Beispiel `String`. So gibt es unter anderem keine Konstruktoren oder Set-Methoden.

Daher werfen wir hier nun einen etwas umfassenderen Blick darauf. Dabei wollen wir vor allem die zugrundeliegenden **Designentscheidungen** (also die Entscheidungen, die die Java-Entwickler getroffen haben, um gut wartbare und intuitiv benutzbare Klassen zur Verfügung zu stellen) beleuchten. Da Sie bereits einige Beispiele von Methodenaufrufen kennen, werden wir hier auf weitere Codelistings verzichten. Sie sind dazu aufgerufen, sich die Benutzung der hier vorgestellten Konzepte selbst zuhause anhand dieses Textes und der API zu erarbeiten.

Alle Klassen zu Datum und Zeit finden Sie im Paket `java.time` (und diversen Unterpaketen). Dieses Paket gibt es seit Java 8. Es macht alle vorherigen Java-Typen zur Datums- und Zeitverarbeitung (wie zum Beispiel `java.util.Date`) überflüssig und verbessert die Benutzung und Verwaltung von Datums- und Zeitangaben an vielen Stellen. Diese neue API basiert auf dem standardisierten Kalendersystem nach ISO 8601 (das ist ein internationaler Standard für Datums- und Zeitangaben), dem der gregorianische Kalender zugrundeliegt. Es ist auch möglich, auf andere Kalendersysteme umzustellen.

Im Paket `java.time` werden **Menschenzeit** und **Maschinenzeit** unterschieden. Menschenzeit wird in Einheiten wie Jahr, Monat, Tag, Minute und so weiter strukturiert und gemessen. Dafür stehen unter anderem die folgenden Klassen zur Verfügung:

- `LocalDate`:
Repräsentiert ein Datum (ohne Zeit) mit Jahr, Monat und Tag.
- `LocalTime`:
Repräsentiert eine übliche Zeit (ohne Datum) mit Stunden, Minuten, Sekunden und Nanosekunden.
- `LocalDateTime`:
Kombination aus Datum und Zeit.
- `Period`:
Dauer zwischen zwei `LocalDate`-Objekten mit Jahr, Monat und Tag.

Diese Klassen repräsentieren Menschenzeit ohne Bezug zu Zeitzonen, dafür gibt es die Klasse `ZonedDateTime`.

Maschinenzeit dagegen wird in Nanosekunden gemessen, beginnend bei einem bestimmten festgelegten Startzeitpunkt, der sogenannten **Epoche**. Dafür gibt es unter anderem die Klassen:

- `Instant`:
Fortlaufende Maschinenzeit in Nanosekunden.
- `Duration`:
Zeitintervall zwischen zwei `Instant`-Objekten.

Die grundlegendste und wichtigste Designentscheidung war, dass alle Datums- und Zeit-Objekte (ebenso wie `String`-Objekte) unveränderlich sind. Dadurch kann es bei der Weitergabe von Objekten (an andere Referenzen) keine Seiteneffekte geben.

Außerdem gibt es eine neue Namensgebung für Methoden. Insbesondere gibt es keine öffentlichen Konstruktoren und Set-Methoden (diese sind für unveränderbare Objekte überflüssig). Stattdessen gibt es Muster, die viele der Klassen aus der neuen API einhalten, unter anderem:

- `now`-Methode (statisch, ersetzt Konstruktor):
Liefert Objekt mit aktueller Zeit / aktuellem Datum.
- `of`-Methoden (statisch, ersetzen Konstruktor):
Erzeugen neue Objekte aus übergebenen Attributwerten.
- `from`-Methoden (statisch, ersetzen Konstruktor):
Erzeugen neue Objekte aus anderen Repräsentationen.
- `parse`-Methoden (statisch, ersetzen Konstruktor):
Erzeugen neue Objekte aus String-Repräsentationen.
- `format`-Methode:
Liefert eine formatierte String-Repräsentation.
- `get`-Methoden:
Lieferten Attributwerte eines Objekts.
- `is`-Methoden:
Fragen Eigenschaften eines Objekts ab.

- **with**-Methoden:
Liefere Kopie des Objekts mit einer geänderten Eigenschaft (ersetze **set**-Methoden).
- **plus**-Methoden:
Liefere Kopie des Objekts mit einer inkrementierten Eigenschaft (ersetze **set**-Methoden).
- **minus**-Methoden:
Liefere Kopie des Objekts mit einer dekrementierten Eigenschaft (ersetze **set**-Methoden).

Interessant ist die formatierte Ausgabe von Zeit- und Datumsobjekten nach vorgegebenen Mustern und Stilen. Die Formatierung erfolgt komplett entkoppelt vom Typ des Objekts und wird durch ein Objekt der Klasse `DateTimeFormatter` erledigt. Für viele gängige Formatierungen bietet diese Klasse vordefinierte Formatierer an, die als öffentliche Klassenattribute zur Verfügung stehen. Nützlich ist auch die Definition eines eigenen Formats mit der Klassenmethode `ofPattern(String pattern)`. Die Beschreibung der Klasse enthält nützlichen Beispielcode zu ihrer Verwendung. Probieren Sie zur Übung verschiedene Ausgabeformate für Datumsangaben aus.

5.3.3 Fallstudie grafischer Editor: Eine unveränderliche Klasse `Point`

Nach den eingangs diskutierten Kernregeln für unveränderliche Klassen, könnte eine unveränderliche Version der Klasse `Point` mit einigen exemplarischen Methoden wie folgt aussehen:

```
public final class Point {

    private final int x;
    private final int y;

    private Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }

    public static Point of(int x, int y) {
        return new Point(x, y);
    }

    public static Point withX(int x) {
        return new Point(x, this.y);
    }

    public static Point plusX(int dx) {
        return new Point(this.x + dx, this.y);
    }
}
```

Hierbei haben wir zugleich die neuen Namensgebungen für Methoden aus der Klasse `LocalDate` übernommen. Es fällt auf, dass der Konstruktor deshalb nun privat ist, denn wir benötigen ihn nur noch intern in den Klassenmethoden zur Erstellung angepasster Kopien.

5.4 Parametrisierte Klassen und Schnittstellen

Klassen (und Schnittstellen) können mit einem Datentyp `T` parametrisiert werden. Durch die Parametrisierung werden Klassen (und Schnittstellen) noch flexibler einsetzbar: Sie werden anwendbar auf jeden Datentyp, den man für `T` einsetzt. Zum Beispiel lassen sich so Sammlungen beliebigen Typs `T` erstellen, also Sammlungen von `String`-Objekten oder von `Person`-Objekten. In diesem Kapitel werden wir in das Konzept der Parametrisierung von Klassen und Schnittstellen einführen und exemplarisch einige wichtige Beispiele aus der Java-API besprechen.

5.4.1 Sammlungen verwalten (`ArrayList<T>`)

Im Unterschied zu C bietet Java vordefinierte Datentypen zur Repräsentation von Arrays variabler Länge an. In C musste wir uns solche Datentypen noch selbst definieren, zum Beispiel dynamische Arrays – das nimmt uns Java also dankenswerterweise ab. Ein solcher Datentyp ist `ArrayList`. Mit einem `ArrayList`-Objekt können wir eine variable (nummerierte) Liste von Objekten verwalten. Wir können zum Beispiel wie folgt eine leere Liste anlegen:¹

```
ArrayList list = new ArrayList();
```

In eine solche Liste können wir mit der `add`-Methode Java-Objekte beliebigen (auch gemischten) Typs aufnehmen, zum Beispiel ein `Integer`- und ein `String`-Objekt:

```
list.add(Integer.valueOf(10));  
list.add("Kartoffelbrei");
```

Neue Objekte werden mit `add` immer an das Ende der Liste gestellt. Nachdem wir einige Objekte aufgenommen haben, wollen wir später natürlich wieder auf diese zugreifen, zum Beispiel um sie auf Kommandozeile auszugeben oder in einer Datei zu speichern. Dafür gibt es eine `get`-Methode, an die wir den Index des Objekts, auf das wir zugreifen wollen, übergeben. Die `get`-Methode gibt dann eine Referenz auf das Objekt zurück. Allerdings ist diese Referenz vom Typ `Object`, denn Java ist typsicher und überprüft schon bei Compilieren, dass alle Datentypen und Methodenaufrufe kompatibel sind. Da wir gemischte Typen in der Liste haben, können wir nur so sicher sein, dass die in der Liste verwalteten Objekte alle vom Typ der Rückgabereferenz sind. Der folgende Aufruf gibt eine Referenz auf den ersten Listeneintrag (dieser hat den Index 0) zurück:

```
Object o = list.get(0);
```

Damit referenziert `o` zwar einen Inhalt vom Typ `Integer`, wir können für `o` aber (erstmal) nur Methoden der Klasse `Object` aufrufen (zum Beispiel `equals` oder `toString`), aber keine speziellen `Integer`-Methoden (wie `intValue`). In vielen Fällen kann das für gemischte Listen auch ausreichend

¹Mit dieser Verwendung sind die meisten Compiler nicht glücklich und reagieren mit einer Warnung – wir werden gleich sehen, warum das so ist und den Code dann verbessern.

sein. Benötigt man jedoch spezielle Methoden der Objekte in der Liste, müsste man die Rückgabereferenz vorher casten:

```
Integer i = (Integer) list.get(0);
```

Aber ist das ein gute Idee?

Für gemischte Listen jedenfalls nicht: Der Typ der hinzugefügten Objekte steht in der Regel erst zur Laufzeit fest (da sie zum Beispiel von Benutzereingaben abhängen). Beim Schreiben des Programms kennen wir also die Typen der später hinzugefügten Objekte noch nicht und wissen damit nicht, welchen Typcast wir benötigen. Kann ein Element nicht auf den gewünschten Typ gecastet werden, so führt dies aber zu einem Programmabbruch. Gemischte Listen machen damit nur dann Sinn, wenn wir für die Objekte in der Liste nur die Methoden einer gemeinsamen Oberklasse wie `Object` aufrufen wollen.

Anders verhält es sich, wenn wir eine Liste brauchen, in die wir nur Objekte eines Typs aufnehmen wollen (zum Beispiel nur `Integer`-Objekte). In solchen Fällen bietet Java sogar eine Möglichkeit, mit der wir komplett auf (die etwas unhandlichen) Typcasts verzichten können. Man kann den Konstruktoraufbau nämlich zusätzlich mit einem Datentyp parametrisieren:

```
ArrayList<Integer> typedList = new ArrayList<Integer>();
```

Dieser Parameter wird direkt hinter den Typ beziehungsweise Konstruktornamen in spitze Klammern geschrieben. Dabei kann man den Typ hinter dem Konstruktornamen auch weglassen (dieser wird vom Compiler dann automatisch abgeleitet):

```
ArrayList<Integer> typedList = new ArrayList<>();
```

In der API sieht der (leere) Konstruktor von `ArrayList` zu diesem Zweck so aus

```
ArrayList<T>()
```

wobei `T` ein Parameter ist, für den ein beliebiger Datentyp eingesetzt werden kann. Die `add`-Methode erlaubt dann nur das Hinzufügen von `T`-Objekten. Dazu hat sie den (ebenfalls) parametrisierten Methodenkopf

```
public boolean add(T o)
```

Es können damit nur Objekte vom Typ `T` übergeben werden, falls man für `T` einen Datentyp eingesetzt hat. In unserem Beispiel haben wir `Integer` für `T` eingesetzt – damit können wir zum Beispiel keine `String`-Objekte mehr hinzufügen. Schließlich ist auch der Methodenkopf der `get`-Methode entsprechend parametrisiert:

```
public T get(int index)
```

Die Rückgabereferenz ist also dann schon vom Typ `T` – wir können auf einen Typcast verzichten und schreiben:

```
Integer i = typedList.get(0);
```

Solche Klassen (und Schnittstellen) mit einem Parameter, für den unterschiedliche Datentypen eingesetzt werden können, heißen **Generics** (zu deutsch **parametrisierte Klassen / Schnittstellen**).

5.4.2 Parametrisierte Klassen

Definition 5.4: Parametrisierte Klasse

Mit der Deklaration

`Klasse<T>`

eines Klassennamens definiert man eine **parametrisierte Klasse mit Parameter `T`**.

- `T` ist ein Parameter, für den ein beliebiger Referenz-Datentyp eingesetzt werden kann
- `T` kann dann als Rückgabetyp und Eingabeparametertyp von Methoden benutzt werden
- Bei Deklaration einer Variable kann man für den Parameter `T` einen konkreten Typ einsetzen.
- Bei Deklaration einer Variable kann man den Parameter `T` auch weglassen. Dann wird für diesen automatisch der Typ `Object` benutzt. Das wird allerdings nicht empfohlen und erzeugt eine Warnung des Compilers.

Die Darstellung in UML sieht wie in Abbildung 1 aus.



Abbildung 1: Eine parametrisierte Klasse in UML.

Somit kann eine parametrisierte Klasse mit beliebigen Datentypen arbeiten – in C hätten wir für jeden Datentyp eine neue Implementierung gebraucht. Gleichzeitig bleibt durch die Parametrisierung die strenge Typprüfung von Java erhalten. Insgesamt ist die Parametrisierung also eine geschickte Abstraktion, ohne Typinformationen zu verlieren.

5.4.3 Sammlungen durchlaufen (`Iterable<T>`)

Ebenso wie Klassen lassen sich auch Schnittstellen parametrisieren. Zum Beispiel implementiert `ArrayList<T>` die parametrisierte Schnittstelle `Iterable<T>` und übernimmt dabei den Parameter `T` von `Iterable<T>`. `Iterable<T>` repräsentiert ganz allgemein **Sammlungen von `T`-Objekten, die der Reihe nach einmal vollständig durchlaufen werden können**. Hierbei ist keine Reihenfolge festgelegt, jedes Element wird *genau einmal* besucht und es ist *nicht* möglich, die Reihenfolge zu

ändern, oder bereits besuchte Elemente noch einmal zu besuchen (weitere implementierende Klassen lernen wir im Unterkapitel 5.5 kennen). Die Schnittstellen-Methode

```
Iterator<T> iterator()
```

liefert einen sogenannten **Iterator** für die Elemente der Sammlung. Wir können uns also wie folgt einen solchen Iterator für unser Beispielliste `typedList` beschaffen:

```
Iterator<Integer> it = typedList.iterator();
```

`Iterator<T>` ist nun selbst wieder eine parametrisierte Schnittstelle. Die Methode `iterator` der Klasse `ArrayList<T>` erstellt also ein Objekt einer (uns verborgenen) Klasse, die die Schnittstelle `Iterator<T>` implementiert, und liefert eine Referenz auf dieses Objekt.

Die Schnittstelle `Iterator<T>`, und damit auch das gelieferte Objekt, stellt folgende Methoden zur Verfügung, um die Sammlung genau einmal zu durchlaufen:

- `T next()`:
Liefert das nächste (noch nicht betrachtete) Objekt (mit parametrisiertem Typ) der Sammlung. Wurden bereits alle Elemente einmal zurückgegeben, führt der Aufruf zu einem Laufzeitfehler.
- `boolean hasNext()`:
Liefert `true`, falls es Elemente in der Sammlung gibt, die noch nicht durchlaufen wurden (die noch nicht von `next` zurückgegeben wurden).

Statt mit einer Schleife den Index zu durchlaufen, können wir damit alternativ den Iterator verwenden, um die Objekte in `typedList` zu durchlaufen:

Beispiel 5.5: Eine Liste ausgeben (ArrayList)

```
ArrayList<Integer> typedList = new ArrayList<>();  
  
// Elemente hinzufügen  
  
Iterator<Integer> it = typedList.iterator();  
while (it.hasNext()) {  
    Integer i = it.next();  
    System.out.println(i);  
}
```

Dieser Code funktioniert ohne Änderung für alle Sammlungen, die `Iterable<T>` implementieren – der Code für das Durchlaufen von Sammlungen ist (durch die Verwendung einer parametrisierten Schnittstelle) **entkoppelt** von der konkreten Implementierung der Sammlung. Das hat den großen Vorteil, dass man später die Implementierung einer Sammlung ohne Seiteneffekte ändern kann.

5.4.4 Objekte vergleichen (Comparable<T>)

Ein weiteres wichtiges Beispiel ist die parametrisierte Schnittstelle `Comparable<T>`. Sie repräsentiert **Objekte, die bezüglich einer natürlichen (totalen) Ordnung verglichen werden können**. Darunter fallen zum Beispiel Hüllklassen für Zahlen (mit der numerischen Ordnung auf Zahlen), der Typ `LocalDate` (mit der üblichen Ordnung auf Datumsangaben) und auch String-Typen (mit der lexikografischen Ordnung). Der Vergleich eines `T`-Objekts `o` mit einem `T`-Objekt `obj` (das als Eingabeparameter übergeben wird) bezüglich dieser Ordnung erfolgt mit der Schnittstellen-Methode

```
int compareTo(T obj)
```

Zum Beispiel implementiert die Klasse `String` die Schnittstelle `Comparable<String>` – hier wird also `String` für den Parameter `T` eingesetzt. Damit können (ausschließlich) `String`-Objekte verglichen werden.

Möchte man Objekte einer eigenen Klasse `MyClass` vergleichen können, so lässt man `MyClass` die Schnittstelle `Comparable<MyClass>` implementieren. Die Implementierung der `compareTo`-Methode definiert dann dazu eine eigene gewünschte Ordnung. Diese Ordnung wird dann von anderen API-Methoden benutzt und **natürliche Ordnung** (dieser Klasse) genannt. Implementiert eine Klasse `T` die Schnittstelle `Comparable<T>`, können beispielsweise Listen von `T`-Objekten mithilfe von API-Methoden nach dieser natürlichen Ordnung sortiert werden, zum Beispiel mit der `sort`-Methode der Klasse `Arrays`.

So könnte man Objekte der Klasse `Person` aus Kapitel 4 wie folgt nach ihrem Geburtsdatum vergleichbar machen:

```
public class Person implements Comparable<Person> {  
  
    private LocalDate geburtsdatum;  
  
    int compareTo(Person p) {  
        return this.getGeburtsdatum().compareTo(p.getGeburtsdatum());  
    }  
}
```

Hier benutzen wir die bereits in der API vorimplementierte Ordnung von `LocalDate`-Objekten – wir **reichen** den Aufruf von `compareTo` für `Person` **durch** an `compareTo` für `LocalDate`. Diese Technik erlaubt, bereits vorhandene Implementierungen wiederzubenutzen, und wird uns an vielen Stellen begegnen.

Bei der von uns gewählten Ordnung für `Person`-Objekte tritt folgende Besonderheit auf: Mehrere `Person`-Objekte können bezüglich `compareTo` gleich groß sein, jedoch bezüglich `equals` verschieden sein. Auf solchen Objekten ist eine Sortierung bezüglich der Ordnung dann undefiniert. Eine solche Ordnung nennt man **nicht konsistent** mit `equals`.

Bei der Definition einer (neuen) natürlichen Ordnung mit `compareTo` sollte man einige Regeln beachten (die allgemein für Ordnungen in der Mathematik gelten):

Definition 5.6: Implementierung der Schnittstelle Comparable<T>

Bei der Implementierung der Schnittstelle Comparable<T> müssen folgende Eigenschaften für beliebige Objekte **x**, **y** und **z** erfüllt sein:

- Der Aufruf **x.compareTo(y)** liefert
 - einen negativen Wert, falls **x** kleiner als **y** ist,
 - den Wert 0, falls **x** und **y** gleich groß sind,
 - einen positiven Wert, falls **x** größer als **y** ist.
- Die Ordnung ist **antisymmetrisch**:
 - $x.compareTo(y) < 0 \Leftrightarrow y.compareTo(x) > 0$
- Die Ordnung ist **transitiv**:
 - $x.compareTo(y) > 0 \wedge y.compareTo(z) > 0 \Rightarrow x.compareTo(z) > 0$
 - $x.compareTo(y) == 0 \Rightarrow (x.compareTo(z) > 0 \Leftrightarrow y.compareTo(z) > 0)$

5.4.5 Parametrisierte Schnittstellen

Definition 5.7: Parametrisierte Schnittstelle

Mit der Deklaration

Schnittstelle <T>

eines Schnittstellennamens definiert man eine **parametrisierte Schnittstelle mit Parameter T**.

- **T** ist ein Parameter, für den ein beliebiger Referenz-Datentyp eingesetzt werden kann.
- **T** kann dann als Rückgabetyt und Eingabeparametertyp von Methoden benutzt werden.
- Bei Implementierung der Schnittstelle oder Deklaration einer Variable kann man für den Parameter **T** einen konkreten Typ einsetzen.
- Bei Implementierung der Schnittstelle oder Deklaration einer Variable kann man den Parameter **T** auch weglassen. Dann wird für diesen automatisch der Typ **Object** benutzt. Das wird allerdings nicht empfohlen und erzeugt eine Warnung des Compilers.

Die Darstellung in UML sieht wie bei parametrisierten Klassen aus (siehe Abbildung 2).



Abbildung 2: Eine parametrisierte Schnittstelle in UML.

5.4.6 Weiterführende Konzepte

Damit haben wir (erst) die einfachen Grundlagen zur Definition und Nutzung von Generics gelegt. Es gibt etliche weiterführende Konzepte, die wir jedoch in dieser Grundlagenveranstaltung nicht genau besprechen können und wollen – insbesondere im Hinblick auf die sinnvolle Nutzung für selbst konzipierte Typen.

Allerdings begegnen uns einige dieser weiterführenden Konzepte im weiteren Verlauf der Vorlesung in der API (sogar schon in diesem Kapitel). Wir wollen uns deshalb abschließend noch zumindest darum kümmern, dass Sie diese korrekt lesen und anwenden können.

Bei den bisherigen Klassen und Schnittstellen mit Parameter `T` muss man bei deren Benutzung einen konkreten Typ einsetzen, der vom Compiler geprüft wird. Daneben gibt es noch die sogenannte **Wildcard** `<?>`, mit der sich ein Datentyp `MyType` parametrisieren lässt. Das Fragezeichen steht für einen bei Übersetzung des Codes noch unbekannten Typ, für den zur Laufzeit ein beliebiger Typ eingesetzt werden muss. Da der Typ bei Übersetzung unbekannt ist, kann man `?` (im Gegensatz zu `T`) nicht als Rückgabotyp oder Eingabeparametertyp von Methoden verwenden, und es lässt sich auch kein Objekt vom Typ `MyType<?>` erzeugen.

Später betrachten wir in der API zwei verschiedene eingeschränkte Varianten von Wildcards, bei denen nur bestimmte Datentypen für `?` eingesetzt werden können:

- **Einschränkung durch eine Obertyp:**

Ist `T` ein Datentyp, so lässt sich ein Datentyp `MyType` wie folgt parametrisieren:

```
MyType<? extends T>
```

In diesem Fall dürfen für `?` nur `T` selbst und Untertypen von `T` eingesetzt werden.

- **Einschränkung durch einen Untertyp:**

Ist `T` ein Datentyp, so lässt sich ein Datentyp `MyType` wie folgt parametrisieren:

```
MyType<? super T>
```

In diesem Fall dürfen für `?` nur `T` selbst und Obertypen von `T` eingesetzt werden.

Beispiele für solche Einschränkungen lernen wir gleich in den folgenden Unterkapiteln kennen.

5.5 Collection-API

Java stellt verschiedene API-Klassen zur Verfügung, mit denen man Sammlungen einer variablen Anzahl von Objekten verwalten kann (sogenannte **Collections**). Bei solchen Sammlungen können Elemente zur Laufzeit beliebig hinzugefügt und gelöscht werden. Die (parametrisierte) Schnittstelle `java.util.Collection<T>` schreibt Verhalten vor, das alle Collections implementieren müssen. Diese Schnittstelle erweitert übrigens die Schnittstelle `Iterable<T>` und mit `ArrayList<T>` haben wir bereits ein Beispiel einer Collection kennengelernt (beides im letzten Unterkapitel 5.4).

5.5.1 Grundlagen

Für alle Collections gilt allgemein:

- Sie können keine Werte primitiven Typs aufnehmen, sondern nur Objekte. Will man primitive Werte aufnehmen, so verwendet man dafür Hüllklassen.
- Alle Collections sind parametrisiert. Setzt man für den Parameter einen konkreten Typ ein, so können nur Objekte dieses Typs verwaltet werden. In diesem Fall spricht man von einer **typisierten Collection**. Lässt man den Parameter weg, so wird automatisch `Object` für den Parameter eingesetzt.

Abbildung 3 zeigt einen Ausschnitt der Collection-API. Es gibt folgende grundsätzliche Arten von Collections:

- **Listen:**

Eine Liste ist eine Sequenz von Elementen, bei der die Elemente eine feste Reihenfolge besitzen und ein Element mehrmals vorkommen darf. Die Schnittstelle `java.util.List<T>` (erweitert `Collection<T>`) schreibt das Verhalten vor, das alle konkreten Listen implementieren müssen. Konkrete Listen sind zum Beispiel:

- `ArrayList`:
Liste auf Basis eines Arrays (schneller Zugriff, langsames Einfügen/Löschen).
- `LinkedList`:
Liste auf Basis einer verketteten Liste (langsamer Zugriff, schnelles Einfügen/ Löschen).

- **Mengen:**

Eine Menge ist eine ungeordnete Sammlung von Elementen, bei der jedes Element nur einmal vorkommen darf. Die Schnittstelle `java.util.Set<T>` (erweitert `Collection<T>`) schreibt das Verhalten vor, das alle konkreten Mengen implementieren müssen. Konkrete Mengen sind zum Beispiel:

- `HashSet`:
Implementierung einer Menge durch Hashing-Verfahren (schnelle Zugriffsoperationen über Schlüsselwerte).
- `TreeSet`:
Implementierung einer Menge durch balancierte Binärbäume (schnelle Zugriffsoperationen über Baumstruktur).

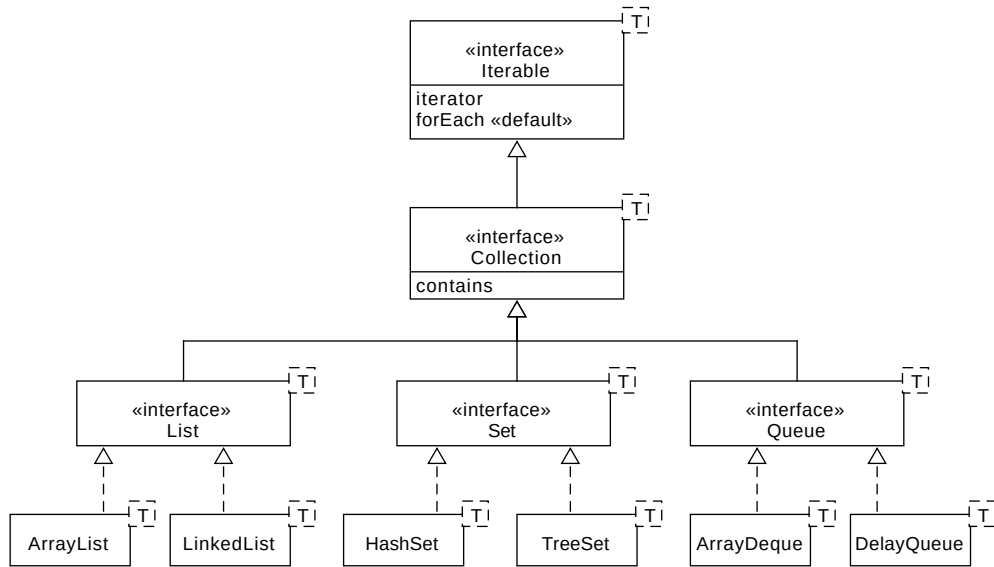


Abbildung 3: API-Ausschnitt der Vererbungs- und Implementierungshierarchie von Collections.

Die Grundlagen zu Hashing und Bäumen werden im Detail in der Vorlesung *Informatik 3* besprochen.

- **Queues (Warteschlangen):**

Eine Queue ist eine spezielle Form einer Liste mit eingeschränkten Zugriffsoperationen, zum Beispiel nach dem FIFO-Prinzip (First In, First Out). Die Schnittstelle `java.util.Queue<T>` (erweitert `Collection<T>`) schreibt das Verhalten vor, das alle konkreten Queues implementieren müssen. Konkrete Queues sind zum Beispiel:

- **ArrayDeque:**
Sehr effiziente Implementierung.
- **DelayQueue:**
Elemente können nur entnommen werden, wenn sie sich lange genug in der Warteschlange befunden haben.

Es gibt also sehr viele verschiedene Arten, wie eine Collection umgesetzt sein kann. Da alle Umsetzungen aber viele gleiche Grundoperationen implementieren, kann man auch im späteren Entwicklungsprozess noch auf eine andere Collection-Implementierung umsteigen, falls diese besser zum Anforderungsprofil passt.

Zum Beispiel kann man für alle Collection-Arten auf die gleiche Art und Weise testen, ob ein Objekt zu einer Collection gehört. Dafür hat `Collection` die Methode

```
boolean contains(Object o)
```

Diese wird von allen Collections implementiert, gibt `true` zurück, wenn `o` in der Collection enthalten ist, und sonst `false`. Dafür werden alle Elemente der Collection durchlaufen und mit `equals`

getestet, ob `o` dem betrachteten Element entspricht. Dies ist ein hervorragendes Beispiel für die Anwendung von Polymorphismus (überlegen Sie!).

Als weiteres Beispiel können alle Collection-Arten auf die gleiche Art und Weise mit einem Iterator durchlaufen werden (zum Beispiel, um alle Elemente auszugeben, oder auch in der eben erwähnten `contains`-Methode). Daher können wir in Beispiel 5.5 `ArrayList` durch `TreeSet` ersetzen, ohne Änderungen am restlichen Code vornehmen zu müssen:

Beispiel 5.8: Ausgabe einer Liste (TreeSet – Version 1)

```
TreeSet<Integer> typedList = new TreeSet<>();

// Elemente hinzufügen

Iterator<Integer> it = typedList.iterator();
while (it.hasNext()) {
    Integer i = it.next();
    System.out.println(i);
}
```

Diese sehr abstrakte Implementierung einer Iteration durch alle Elemente liegt übrigens daran, dass gewisse Collections (wie zum Beispiel Mengen) nicht unbedingt eine Reihenfolge implementieren und ihre Elemente nicht zwingend einen Index haben, den man durchlaufen könnte. Deshalb ist die Reihenfolge, mit der ein Iterator eine Collection durchläuft, undefiniert und kann nicht beeinflusst werden. Da der obige Code etwas umständlich ist, gibt es dafür eine abgekürzte Schreibweise, die in unserem Beispiel wie folgt aussieht:

Beispiel 5.9: Ausgabe einer Liste (TreeSet – Version 2)

```
TreeSet<Integer> typedList = new TreeSet<>();

// Elemente hinzufügen

for (Integer i : typedList) {
    System.out.println(i);
}
```

Diese Schleife nennt man eine `for-each`-Schleife. Diese ist allgemein wie folgt definiert:

Definition 5.10: for-each-Schleife

Implementiert `collection` die Schnittstelle `Iterable<T>` oder ist `collection` ein `T`-Array, dann kann man `collection` mit

```
for (T o : collection) {  
    // do something for o  
}
```

genau einmal vollständig durchlaufen. Hierbei ist `o` ein frei wählbarer Variablenname für das aktuell in einem bestimmten Schleifendurchlauf betrachtete Element in `collection`.

Bei Arrays führt die for-each-Schleife intern zum selben Byte-Code wie die Ihnen bisher bekannte for-Schleife.

5.5.2 Fallstudie grafischer Editor: Container-Klassen

In diesem Unterkapitel wollen wir eine Anwendung von Sammlungen wie `ArrayList<T>` und der Schnittstelle `Iterable<T>` für selbst erstellte Datenklassen diskutieren. Dazu erweitern wir unsere bisherige Modellierung und Implementierung des grafischen Editors.

In einer Anwendung müssen wir die Möglichkeit haben, systematisch auf alle zur Laufzeit erstellten Datenobjekte zugreifen zu können. Vielleicht wollen wir Datenobjekte sortieren, durchsuchen oder ausgeben. Oder im Fall unseres grafischen Editors zeichnen. Üblicherweise erstellt man zu diesem Zweck zu einer Datenklasse eine zugehörige sogenannte **Containerklasse**, in der man alle zur Laufzeit erzeugten Objekte der Datenklasse verwaltet. Dazu stellt man in der Containerklasse Methoden zum Hinzufügen von Objekten, zum Löschen von Objekten und für den Zugriff auf alle Objekte der Datenklasse zur Verfügung. Dieses Vorgehen nennt man **Containermuster** – ein Software-Entwurfsmuster in der objektorientierten Programmierung. Das sieht in UML für die Datenklasse `Form` des grafischen Editors wie in Abbildung 4 aus.

Neu in Abbildung 4 sind die Klasse `FormContainer` und die Schnittstelle `Iterable` (die restlichen Klassen und Schnittstellen kennen wir schon aus Kapitel 4 und bleiben unverändert):

- Der Pfeil mit Raute von der Klasse `FormContainer` zur Klasse `Form` repräsentiert eine *besteht aus*-Beziehung zwischen Objekten dieser beiden Klassen: Ein `FormContainer`-Objekt *besteht aus* `Form`-Objekten. Umgekehrt gilt: Ein `Form`-Objekt *ist Teil eines* `FormContainer`-Objekts. Ein `FormContainer`-Objekt dient zu Verwaltung der `Form`-Objekte, aus denen es besteht. Die Beschriftung `allForms` an der Pfeilspitze repräsentiert die Referenz, über die man in einem `FormContainer`-Objekt auf diese `Form`-Objekte zugreifen kann.
- Zur Verwaltung der `Form`-Objekte dienen drei Methoden des `FormContainer`-Objekts:
 - Die `addForm`-Methode fügt dem Container eine zusätzliche `Form` hinzu.
 - Die `removeForm`-Methode löscht eine `Form` aus dem Container.

Die Referenz `allForms` aus dem UML-Modell implementieren wir als Attribut des Containers. Als Datentyp für dieses Attribut verwenden wir mit `ArrayList<Form>` eine API-Collection, da wir beliebig viele Formen verwalten wollen. Bei Erstellung eines Container-Objekts wird `allForms` intern als leere Liste initialisiert. Wird zur Laufzeit ein Kreis oder Rechteck erzeugt, übergeben wir diese Form an die Methode `addForm`. Diese reicht die Form durch an die `add`-Methode von `allForms` (siehe API). Diese nimmt die Form so in die interne Liste auf. Wollen wir zur Laufzeit eine Form löschen, übergeben wird diese an die Methode `removeForm`. Diese reicht die Form durch an die `remove`-Methode von `allForms` (siehe API). Diese löscht die Form aus der internen Liste.

Zudem implementiert der Container die Schnittstelle `Iterable<Form>`. Dadurch verpflichten wir unseren Container dazu, die Schnittstellen-Methode `iterator` zu implementieren. Diese muss laut Spezifikation ein `Iterator<Form>`-Objekt zurückgeben. Durch die Tatsache, dass das `ArrayList`-Objekt `allForms` schon die Methode `iterator` zur Verfügung stellt, können wir unsere Container-Klasse leicht mit einer ebensolchen Methode ausstatten:

Wir geben einfach den Iterator von `allForms` zurück.

Dadurch können wir alle Formen der internen Liste `allForms` von außen durchlaufen. Das geht dann genauso, wie wir das im vorherigen Unterkapitel gelernt haben.

Damit können wir in etwa wie folgt einen Container und einige grafische Objekte erstellen, die grafischen Objekte in den Container aufnehmen, und alle aktuell vorhandenen Objekte durchlaufen und ausgeben:

```
FormContainer con = new FormContainer();

con.addForm(new Circle(50, new Point(10, 20)));
con.addForm(new Rectangle(10, 10, new Point(20, 10)));

for (Form f : con) {
    System.out.println(f);
}
```

Hier wird jeweils ein sogenanntes **anonymes Objekt** (das heißt ein Objekt, auf das keine explizite Referenz vorhanden ist) vom Typ `Circle` und `Rectangle` erzeugt und übergeben. Es gibt dann nur eine implizite Referenz auf diese beiden Objekte im Container-Objekt `con` (nämlich in der internen Sammlung `allForms`). Durch die Anwendung des Iteratormusters können wir unseren Container implementierungsunabhängig einheitlich mit einer `for-each`-Schleife durchlaufen. In der Schleife können wir dabei nur Methoden der Klasse `Form` auf `f` anwenden. Beispielweise können wir alle Formen in ihrer Position verschieben (`f.moveRight()`).

Die `add`- und `remove`-Methode des Containers sind dabei weitere Beispiele für das Substitutionsprinzip. Beide Methoden erwarten ein Argument vom Typ `Form`. Zur Laufzeit können wir für diesen Typ Objekte vom Unterklassen-Typ `Circle` und `Rectangle` einsetzen. Dadurch benötigen wir auch für Kreise und Rechtecke keine eigenen Container zur Verwaltung.

Sie stellen sich nun vielleicht die Frage, ob es wirklich sinnvoll und notwendig ist, eine eigene Containerklasse zur Verwaltung aller Formen anzulegen. Alternativ könnten wir es uns ja auch

viel einfacher machen, und direkt eine der bereits vorhandenen API-Collectionklassen verwenden, in etwa so:

```
ArrayList<Form> list = new ArrayList<Form>();

list.add(new Circle(50, new Point(10, 20)));
list.add(new Rectangle(10, 10, new Point(20, 10)));

for (Form f : list) {
    System.out.println(f);
}
```

Denn sowieso verwenden wir im Container intern eine solche API-Collectionklasse und reichen alle Anforderungen wir das Hinzufügen und Löschen von Formen an diese durch. Ein solches vereinfachtes Vorgehen hätte aber die folgenden Nachteile:

- Der Container `con` lässt nur einen speziell auf die Anforderungen an die Anwendung angepassten Zugriff auf seine Formen zu, nämlich über `addForm`, `removeForm` und den Iterator. Die Liste `list` stellt dagegen noch viel mehr Zugriffsmethoden zur Verfügung, von denen wir die meisten zur Verwaltung der Formen gar nicht benötigen (sehen Sie sich dazu die API an). Diese müssten wir zudem erstmal alle überprüfen, ob sie keine unserer Anforderungen an die Anwendung verletzen. Besser ist es, nur genau die Methoden zur Verfügung zu stellen, die man auch benötigt.
- Der Container `con` erlaubt es uns, später weitere spezifische Methoden hinzuzunehmen, die noch benötigt werden. So werden später zum Beispiel noch Methoden zum Laden und Abspeichern von Daten hinzukommen. Die Liste `list` könnten wir nur erweitern, indem wir eine Unterklasse von `ArrayList` bilden.
- Der Container `con` erlaubt es uns, später problemlos die intern verwendete Collectionklasse auszutauschen, falls eine andere besser geeignet ist. Der restliche Code in anderen Klassen muss dabei nicht geändert werden. Die Liste `list` könnten wir hingegen nur unter großem Aufwand durch eine andere Collectionklasse ersetzen, denn dazu müssten wir in allen anderen Klassen die verwendeten Referenzen auf `list` anpassen und alle Methodenaufrufe auf `list` überprüfen.

Abschließend noch eine Anmerkung zu einem Unterschied zwischen dem UML-Modell und der Java-Implementierung. Vielleicht ist Ihnen aufgefallen, dass `allForms` zwar ein Attribut der Java-Klasse, aber nicht der UML-Klasse ist. Das ist kein Versehen, denn im UML-Modell steht `allForms` stattdessen als Referenz am *besteht aus*-Pfeil. Da wäre es erstens redundant, `allForms` zusätzlich noch als Attribut in die UML-Klasse aufzunehmen. Zweitens wäre es weniger informativ, `allForms` stattdessen als Attribut in die UML-Klasse aufzunehmen, denn dadurch ginge die Information verloren, dass sich `allForms` auf die Klasse `Form` bezieht. Die UML-Darstellung der Beziehung zwischen Container und enthaltenen Objekten über einen *besteht aus*-Pfeil mit Referenz statt über ein Attribut (wie in Java) ist grafisch wesentlich übersichtlicher und informativer.

5.5.3 Die `forEach` - Methode

Eine sehr praktische Funktionalität von Iteratoren ist die Möglichkeit, mit einer einheitlichen Schleife eine ausgewählte Operation auf alle Elemente einer Collection anzuwenden. Dazu besitzt

die Schnittstelle `Iterable<T>` die folgende Default-Methode:

```
default void forEach(Consumer<? super T> action) {
    for (T t : this)
        action.accept(t);
}
```

Dabei ist `Consumer<T>` eine Schnittstelle mit einer Methode

```
void accept(T t)
```

zur Ausführung einer Operation auf einem `T`-Objekt `t` ohne Rückgabewert.

Mit `this` wird auf das `Iterable`-Objekt selbst, also die Sammlung, für die `forEach` aufgerufen wird, verwiesen. Die `forEach`-Methode führt also die `accept`-Methode eines (an `forEach` übergebenen) `Consumer`-Objekts auf allen Elementen einer Collection aus.

In obiger Benutzung kann durch die Wildcard-Parametrisierung, wie im letzten Unterkapitel beschrieben, für den Typ `Consumer<? super T>` ein `Consumer`-Objekt eingesetzt werden, dessen Parameter ein Obertyp von `T` (oder `T` selbst) ist.

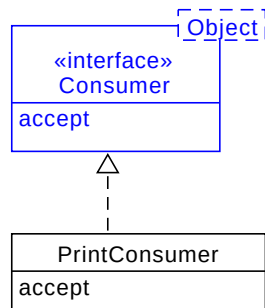


Abbildung 5: Eine eigene `Consumer<Object>`-Klasse.

Als eine sehr einfache Anwendung wollen wir nun so auf eine weitere Art alle Elemente einer beliebigen parametrisierten Collection mit `println` ausgeben. **Ein dafür passendes `Consumer`-Objekt mit einem passenden Parameter können wir uns wie folgt beschaffen:**

- Der passende Parameter ist hier `Object`, denn `Object` ist ein Obertyp von `T` (für jedes beliebige `T`), und `Object`-Objekte können an `println` übergeben werden.
- Das `Consumer`-Objekt erhalten wir als Objekt einer passend von uns neu definierten Klasse, die `Consumer<Object>` implementiert, und die wir zum Beispiel `PrintConsumer` nennen können.

Abbildung 5 illustriert unser Vorgehen in UML. Unsere selbst definierte Klasse `PrintConsumer` ist hierbei schwarz dargestellt und die API-Schnittstelle in blau hervorgehoben. Diese Klasse enthält nur die Methode `accept` zur Implementierung der Schnittstelle. Da keine Daten verwaltet werden, reicht ein leerer Konstruktor, und der ist automatisch vorhanden, ohne ihn angeben zu müssen. Beispiel 5.11 zeigt die zugehörige Implementierung.

Beispiel 5.11: Einen Consumer-Typ für Ausgaben definieren

```
public class PrintConsumer implements Consumer<Object> {
    public void accept(Object t) {
        System.out.println(t);
    }
}
```

Mit dem leeren Konstruktor können wir ein `PrintConsumer`-Objekt erstellen und an `forEach` eines Objekts vom Typ `TreeSet<Integer>` übergeben. Die Klasse `PrintConsumer` implementiert `Consumer<Object>` und `forEach` akzeptiert nach dem Substitutionsprinzip einen Eingabeparameter von diesem Typ, da `Object` ein Obertyp von `Integer` ist. Beispiel 5.12 zeigt die zugehörige Implementierung und damit eine weitere Möglichkeit der Ausgabe der Elemente einer Collection.

Beispiel 5.12: Ausgabe einer Liste (TreeSet – Version 3)

```
TreeSet<Integer> typedList = new TreeSet<Integer>();

// Elemente hinzufügen

typedList.forEach(new PrintConsumer());
```

Hier wird ein anonymes Objekt vom Typ `PrintConsumer` erzeugt und übergeben. Damit gibt es nur eine lokale implizite Referenz auf das `PrintConsumer`-Objekt in der `forEach`-Methode, die nach Abarbeitung der Methode nicht mehr existiert. Dann brauchen wir das Objekt auch nicht mehr und es fällt der Garbage Collection zum Opfer. Die `forEach`-Implementierung wendet die `accept`-Methode dieses Objekts auf jedes Element der Collection an.

Damit ist der Code noch etwas kompakter geworden im Vergleich zu einer `for-each`-Schleife, allerdings zum Preis einer zusätzlich erstellten Klasse. Aber wir können auch ganz anderen Dinge mit dieser Technik anstellen. Zum Beispiel können wir mit dem `Consumer` aus Beispiel 5.13 jedes Element einer beliebigen mit `StringBuilder` parametrisierten Collection auf eine bestimmte Länge begrenzen:

Beispiel 5.13: Definition eines Consumer-Typs für Längenbegrenzung von Strings

```
public class CutConsumer implements Consumer<StringBuilder> {
    public void accept(StringBuilder s) {
        s.setLength(5);
    }
}
```

Und natürlich können wir dieses Prinzip auch auf Containerklassen wie im letzten Unterkapitel anwenden und damit beispielsweise die Positionen aller grafischen Objekte versetzen:

Beispiel 5.14: Definition eines Consumer-Typs zum Versetzen grafischer Objekte

```
public class MoveConsumer implements Consumer<Form> {  
    public void accept(Form f) {  
        f.moveRight();  
    }  
}
```

Das ist jetzt alles schon ziemlich cool, wenn da nicht ständig die umständliche Erstellung eines neuen `Consumer`-Typs wäre. Dies sorgt im Zweifelsfall zudem für eine unübersichtliche Projektstruktur, da für die Verarbeitung von Collections zahlreiche kleine Klassen erstellt werden müssen. Hier werden uns **Lambda-Ausdrücke** helfen – mit diesen lassen sich Typen und Objekte bestimmter Schnittstellen (wie `Consumer`) äußerst kompakt und nach etwas Übung sehr intuitiv erstellen. Diese gibt es seit Java 8 und sind Thema des folgenden Unterkapitels 5.6.

Wir fassen an dieser Stelle nochmal mögliche allgemeine Vorgehensweisen zusammen, wenn eine Methode (wie `forEach`) einen Eingabeparamter verlangt, der vom Typ einer Schnittstelle ist (wie `Consumer<T>`).

Definition 5.15: Eingabeparameter mit Schnittstellen-Typ benutzen

Hat eine Methode einen Eingabeparameter vom Typ einer Schnittstelle `S`, so können wir auf folgende Weisen ein Objekt erzeugen, das wir beim Methodenaufruf als Argument übergeben können:

1. Erzeuge ein Objekt einer Klasse, die `S` implementiert, und übergib dieses bei Methodenaufruf.
2. Erweitere eine bereits existierende Klasse, so dass diese `S` implementiert. Dann erzeuge ein Objekt dieser Klasse und übergib dieses bei Methodenaufruf.
3. Erstelle eine neue Klasse, die `S` implementiert. Dann erzeuge ein Objekt dieser Klasse und übergib dieses bei Methodenaufruf.

Beispiel: Klasse `PrintConsumer`.

Überlegen Sie abschließend anhand der besprochenen Beispiele, warum es sinnvoll ist, dass der Methodenkopf von `forEach` die Verwendung eines Obertyps des `Iterable`-Parameters `T` für `Consumer` erlaubt. Wieso wird eigentlich nicht einfach `T` durchgereicht?

5.6 Lambda-Ausdrücke

Wir wollen nun eine sehr kompakte Möglichkeit kennenlernen, Typen und Objekte bestimmter Schnittstellen anzulegen. Eine solche Schnittstelle, für die das funktioniert, ist die `Consumer`-Schnittstelle, die wir schon ausführlich im letzten Unterkapitel 5.5 besprochen haben. Für diese können wir den Code aus den Beispielen 5.11 und 5.12 ersetzen durch:

Beispiel 5.16: Ausgabe einer Liste (TreeSet – Version 4)

```
TreeSet<Integer> typedList = new TreeSet<Integer>();  
  
// Elemente hinzufügen  
  
typedList.forEach((Object t) -> {System.out.println(t  
    );});
```

Wir sparen uns also die komplette Definition des Typs `PrintConsumer`, und übergeben anstatt eines neuen `PrintConsumer`-Objekts den Ausdruck

```
(Object t) -> {System.out.println(t);}
```

an `forEach` – kürzer und übersichtlicher geht es kaum (später lernen wir eine noch etwas kompaktere Schreibweise dieses Ausdruck kennen). Dieser Ausdruck ist ein sogenannter **Lambda-Ausdruck**. Er ist eine sehr kompakte Möglichkeit, ein `Consumer`-Objekt zu erstellen, das sich genauso verhält wie das `PrintConsumer`-Objekt aus dem letzten Unterkapitel. Dazu implementiert der Lambda-Ausdruck wie folgt die `accept`-Methode eines `Consumer`-Objekts: Vor dem Pfeil steht in runden Klammern die Liste der Eingabeparameter, und hinter dem Pfeil steht in geschweiften Klammern die Implementierung von `accept`. Nun haben Sie bestimmt Fragen über Fragen, denn hier fehlen doch nun wirklich einige Informationen:

Woher weiß nun der Compiler, dass er den Lambda-Ausdruck einer Methode der `Consumer`-Schnittstelle zuordnen soll, und nicht irgendeiner anderen Schnittstelle?

Da (der Methodenkopf von) `forEach` ein `Consumer`-Objekt erwartet, versucht der Compiler, den Lambda-Ausdruck der `Consumer`-Schnittstelle zuzuordnen. Das nennt man **Typinferenz**.

Und woher weiß der Compiler, dass er den Lambda-Ausdruck der `accept`-Methode der `Consumer`-Schnittstelle zuordnen soll und nicht irgendeiner anderen Methode?

Die `Consumer`-Schnittstelle hat nur eine zu implementierende Methode, nämlich `accept`, also kann nur diese gemeint sein. Solche Schnittstellen heißen **funktional**, und nur für solche lassen sich Lambda-Ausdrücke definieren. Deshalb betrachten wir solche Schnittstellen nun ausführlich.

5.6.1 Funktionale Schnittstellen

Wir wollen uns zuerst ein weiteres Beispiel ansehen für die Verwendung von Lambda-Ausdrücken als Eingabeparameter für funktionale Schnittstellen. Zum Beispiel können wir wie folgt aus einem `ArrayList<Form>`-Objekt `list` alle Formen entfernen, deren X-Position kleiner als 100 ist:

```
list.removeIf((Form f) -> {return f.getXPosition() < 100;});
```

Dazu benutzen wir die `removeIf`-Methode. Diese erwartet als Eingabeparameter ein `Predicate<Form>`-Objekt. `Predicate<Form>` ist eine funktionale Schnittstelle mit genau einer abstrakten Methode, nämlich:

```
boolean test(Form f)
```

Die `removeIf`-Methode entfernt aus `list` alle Formen `f`, für die `test` den Wert `true` liefert. Mit dem Lambdaausdruck

```
(Form f) -> {return f.getXPosition() < 100;}
```

definieren wir die Implementierung von `test`. Links vom Pfeil stehen mit

```
(Form f)
```

die Eingabeparameter von `test`. Rechts vom Pfeil steht mit

```
{return f.getXPosition() < 100;}
```

der Rumpf von `test`.

Damit haben wir bereits einige funktionale Schnittstellen kennengelernt, neben `Consumer` zum Beispiel `Iterable`, `Comparable` und gerade eben `Predicate`. `Consumer` und `Predicate` gehören dabei zu einem ganzen Paket mit nützlichen, weil sehr allgemein und generisch einsetzbaren, funktionalen Schnittstellen, das es seit Java 8 gibt.

Definition 5.17: Funktionale Schnittstellen

Eine **funktionale Schnittstelle** ist eine Schnittstelle mit **genau einer abstrakten (also zu implementierenden) Methode**. Zusätzlich zu der einzigen abstrakten Methode darf die Schnittstelle noch statische oder Default-Methoden besitzen, für die schon eine Implementierung existiert. Die funktionalen Schnittstellen aus dem Paket `java.util.function` realisieren mit ihrer abstrakten Methode bestimmte Typen von Funktionen, also Abbildungen der Form

$\langle \text{Eingabeparametertypen} \rangle \rightarrow \langle \text{Rückgabetypp} \rangle$

Einige Beispiele sind:

- `Consumer<T>` (Verarbeitung eines T-Objekts):
Realisiert eine Abbildung der Form
 $(T) \rightarrow \text{void}$
durch Ausführung der Operation `void accept(T t)`.
Beispiel: Verwendung in der API in der `forEach`-Methode einer Collection
- `Supplier<T>` (Generierung eines T-Objekts):
Realisiert eine Abbildung der Form
 $() \rightarrow T$
durch Ausführung der Operation `T get()`.
Beispiel: Verwendung in der API in der `generate`-Methode eines Streams (siehe nächstes Unterkapitel 5.7)
- `Predicate<T>` (Testen einer Eigenschaft eines T-Objekts):
Realisiert eine Abbildung der Form
 $(T) \rightarrow \text{boolean}$
durch Ausführung der Operation `boolean test(T t)`.
Beispiel: Verwendung in der API in der `removeIf`-Methode einer Collection
- `Function<T, R>` (Umwandlung eines T-Objekts in ein R-Objekt):
Realisiert eine Abbildung der Form
 $(T) \rightarrow R$
durch Ausführung der Operation `R apply(T t)`.
Beispiel: Verwendung in der API in der `map`-Methode eines Streams (siehe nächstes Unterkapitel 5.7)
- `UnaryOperator<T>` (Umwandlung eines T-Objekts in ein anderes T-Objekt):
Realisiert eine Abbildung der Form
 $(T) \rightarrow T$
durch Ausführung der Operation `T apply(T t)`.
Beispiel: Verwendung in der API in der `iterate`-Methode eines Streams (siehe nächstes Unterkapitel 5.7)

Mit einem Lambda-Ausdruck lässt sich ein Objekt vom Typ einer funktionalen Schnittstelle erstellen. Allgemein sind Lambda-Ausdrücke wie folgt definiert:

Definition 5.18: Lambda-Ausdruck

Ein **Lambda-Ausdruck** zu einer funktionalen Schnittstelle **S** ist von der Form:

`(<LambdaParameter>) -> {<Anweisungen>}`

- Auf der *linken Seite* steht eine durch Kommas getrennte Liste der Eingabeparameter der abstrakten Schnittstellenmethode.
- Auf der *rechten Seite* befindet sich die Implementierung der Schnittstellenmethode.
- Ein Lambda-Ausdruck zu **S** ist ein **S**-Objekt.

Erwartet also eine Methode als Eingabeparameter ein Objekt vom Typ einer funktionalen Schnittstelle, so kann für diesen Parameter ein Lambda-Ausdruck (zu der funktionalen Schnittstelle) eingesetzt werden. Dabei definiert der Lambda-Ausdruck den Code der (einzigen abstrakten) Schnittstellenmethode. Außerdem kann man ein Objekt vom Typ einer funktionalen Schnittstelle durch einen Lambda-Ausdruck erstellen. Zum Beispiel kann man wie folgt ein **Consumer**-Objekt **cut**, das sich wie der **CutConsumer** aus Beispiel 5.13 verhält, erstellen:

```
Consumer<StringBuilder> cut = (StringBuilder s) -> {s.setLength(5);};
```

In bestimmten Fällen gibt es noch einige vereinfachende und kompaktere Schreibweisen, bei denen weitere Informationen weggelassen werden können, die sich der Compiler selbstständig erschließen kann:

1. Können die Typen der Eingabeparameter aus dem Methodenkopf der Schnittstellenmethode abgeleitet werden, müssen diese nicht angegeben werden.

```
lang: Consumer<StringBuilder> cut = (StringBuilder s) -> {s.setLength(5);};
```

```
kurz: Consumer<StringBuilder> cut = (s) -> {s.setLength(5);};
```

2. Besteht die rechte Seite nur aus **einer** Anweisung, so können die geschweiften Klammern und der Strichpunkt weggelassen werden.

```
lang: Consumer<StringBuilder> cut = (s) -> {s.setLength(5);};
```

```
kurz: Consumer<StringBuilder> cut = (s) -> s.setLength(5);
```

3. Besteht die Eingabeparameterliste nur aus **einem** Parameter, so können die runden Klammern weggelassen werden.

```
lang: Consumer<StringBuilder> cut = (s) -> s.setLength(5);
```

```
kurz: Consumer<StringBuilder> cut = s -> s.setLength(5);
```

4. Besteht die rechte Seite nur aus **einer** **return**-Anweisung, so darf man **return** weglassen.

```
lang: Supplier<Double> rand = () -> {return Math.random();};
```

```
kurz: Supplier<Double> rand = () -> Math.random();
```

5.6.2 Methoden- und Konstruktorreferenzen

Außerdem gibt es noch zwei völlig andere Schreibweisen für Lambda-Ausdrücke: **Methodenreferenzen** und **Konstruktorreferenzen**. Bei diesen wird als Schnittstellenmethode eine geeignete bereits vorhandene Methode oder ein geeigneter bereits vorhandener Konstruktor verwendet.

Eine **Methodenreferenz** gibt es in 3 verschiedenen Formen:

1. Besteht die rechte Seite eines Lambda-Ausdrucks nur aus dem Aufruf **einer Klassenmethode** `<Klassenmethode>`, deren Parameter und Rückgabetyt mit denen der Schnittstellenmethode übereinstimmen, so darf man statt

`(<Parameter>) -> <Klassenname>.<Klassenmethode>(<Parameter>)`

eine sogenannte **Methodenreferenz** auf diese Methode in der Form

`<Klassenname>::<Klassenmethode>`

verwenden.

kurz: `Supplier<Double> rand = () -> Math.random();`

noch kürzer: `Supplier<Double> rand = Math::random;`

(Überlegen Sie warum dieser Code korrekt ist, obwohl `double` und nicht `Double` der Rückgabetyt von `random` ist)

2. Besteht die rechte Seite eines Lambda-Ausdrucks nur aus dem Aufruf **einer Objektmethode** `<Objektmethode>`, deren Parameter und Rückgabetyt mit denen der Schnittstellenmethode übereinstimmen, so darf man statt

`(<Parameter>) -> <Objekt>.<Objektmethode>(<Parameter>)`

eine sogenannte **Methodenreferenz** auf diese Methode in der Form

`<Objekt>::<Objektmethode>`

verwenden.

kurz: `Consumer<Integer> print = t -> System.out.println(t);`

noch kürzer: `Consumer<Integer> print = System.out::println;`

3. Gibt es nur einen Parameter `p` vom Typ `T` und besteht die rechte Seite nur aus dem Aufruf **einer Objektmethode** `<Objektmethode>` für `p`, die keine Eingabeparameter hat und deren Rückgabetyt mit dem der Schnittstellenmethode übereinstimmt, so darf man statt

`p -> p.<Objektmethode>()`

eine sogenannte **Methodenreferenz** auf diese Methode in der Form

`T::<Methode>`

verwenden (und es sind auch Varianten mit mehr als einem Parameter möglich).

kurz: `Consumer<Form> move = f -> f.moveRight();`

noch kürzer: `Consumer<Form> move = Form::moveRight;`

Bleiben abschließend **Konstruktorreferenzen**. Hier gibt es nur eine Variante. Besteht die rechte Seite eines Lambda-Ausdrucks nur aus dem Aufruf **eines Konstruktors** `<Konstruktor>`, dessen Parameter und Rückgabotyp mit denen der Schnittstellenmethode übereinstimmen, so darf man statt

```
( <Parameter> ) -> new <Konstruktor> ( <Parameter> )
```

eine sogenannte **Konstruktorreferenz** auf diesen Konstruktor in der Form

```
<Konstruktor>::new
```

verwenden.

```
kurz: Function<String, StringBuilder> convert = s -> new StringBuilder(s);
```

```
noch kürzer: Function<String, StringBuilder> convert = StringBuilder::new;
```

Fassen wir noch einmal zusammen: In der Java-API gibt es einige sehr nützliche Anwendungen sogenannter funktionaler Schnittstellen – zum Beispiel um Methoden auf Collections auszuführen. Oft benötigt man in einer Anwendung eine bestimmte solche Methode nur einmal an einer bestimmten Stelle und viele verschiedene solche Methoden. Mit Lambda-Ausdrücken haben wir eine sehr kompakte Möglichkeit kennengelernt, Instanzen solcher Schnittstellen zu erstellen, ohne den „Umweg“ gehen müssen, für jede Methode eine eigene separate Klasse definieren zu müssen. Mit einem Lambda-Ausdruck setzen wir für einen Eingabeparameter direkt die Methode ein, statt einem Objekt. Das hat auch den Vorteil, dass man an Ort und Stelle im Code sehen kann, welche Methode dort ausgeführt werden soll, ohne dass man das in einer anderen Klasse nachsehen muss (wenn man ein Objekt übergeben würde). Das macht den Code verständlicher und übersichtlicher.

Allerdings sollte man die Wiederverwendbarkeit und Wartbarkeit von Code im Blick behalten. Benötigt man eine Methode sehr oft, und verwendet man an allen Stellen Lambda-Ausdrücke für die Methode, so entsteht viel redundanter und schlecht wartbarer Code (denn eine Änderung der Methode müsste an vielen Stellen durchgeführt werden). In einem solchen Fall kann es besser sein, eine eigene Klasse für die Methode zu definieren, und Objekte dieser Klasse zu übergeben. Dann existiert der Code der Methode nur genau einmal (nämlich in dieser Klasse), und ist nicht über viele Stellen im Code verstreut (nämlich in jedem der Lambda-Ausdrücke).

Weitere schöne Beispiele, Anwendungen und Einsatzmöglichkeiten für Lambda-Ausdrücke werden Ihnen im Screencast *Methoden-in-einer-Nussschale-Lambda-Ausdrücke-in-Java* nahegebracht.

Ein weiteres wichtiges Anwendungsfeld für funktionale Schnittstellen und Lambda-Ausdrücke sind Streams im folgenden Unterkapitel 5.7.

5.7 Streams

Neben den in den letzten beiden Unterkapiteln diskutierten Möglichkeiten zum Durchlaufen von Collections gibt es noch eine weitere Möglichkeit durch Umwandlung einer Collection in einen **Stream**.

Das hat unter anderem den Vorteil, dass für Streams noch zusätzliche nützliche Möglichkeiten existieren, Operationen auf ihren Elementen auszuführen: Beispielsweise gibt es neben Methoden zum Filtern (also dem Entfernen bestimmter Elemente) auch Methoden zur Umwandlung und zum Sortieren von Elementen. Außerdem können mehrere solche Methoden kompakt direkt hintereinander in einer Anweisung ausgeführt werden. Und es ist möglich, Aggregationen wie zum Beispiel Summenbildung und Zählen der Anzahl der Elemente durchzuführen.

Nehmen wir zum Beispiel an, wir wollen als Kommandozeilenparameter übergebene ganze Zahlen sortieren und dann alle Zahlen, die kleiner sind als 10, ausgeben. Das geht in einer Anweisung wie folgt:

```
Arrays.stream(args)
    .map(Integer::parseInt)
    .sorted()
    .filter(n -> n < 10)
    .forEach(System.out::println);
```

Hier passiert der Reihe nach Folgendes:

1. Die `stream`-Methode von `Arrays` wandelt das `String`-Array `args` in ein `Stream<String>`-Objekt um und gibt dieses zurück.
2. Mit der `map`-Methode von `Stream` kann man die Elemente eines Streams umwandeln. Die Umwandlung wird definiert durch ein übergebenes `Function`-Objekt. `Function` ist eine funktionale Schnittstelle, für die hier der Lambda-Ausdruck `Integer::parseInt` übergeben wird. Diese Methode wandelt `String`-Objekte in ganze Zahlen um. Mit Autoboxing bekommen wir so als Ergebnis einen `Integer`-Stream. Überprüfen Sie, dass hier die Regeln für Methodenreferenzen erfüllt sind!
3. Die `sorted`-Methode von `Stream` sortiert die Elemente bezüglich ihrer natürlichen Ordnung (hier definiert durch die Implementierung der `Comparable`-Schnittstelle der Klasse `Integer`).
4. Mit der `filter`-Methode von `Stream` kann man die Elemente eines Streams bezüglich einer Eigenschaft filtern – nur die Elemente, die diese Eigenschaft erfüllen, bleiben im Stream (alle anderen werden aus dem Stream entfernt). Die Eigenschaft wird definiert durch ein übergebenes `Predicate`-Objekt. `Predicate` ist eine funktionale Schnittstelle, für die hier der Lambda-Ausdruck `n -> n < 10` übergeben wird – es bleiben also nur Zahlen `n`, die kleiner sind als 10, im Stream.
5. Wie Collections haben Streams eine `forEach`-Methode, die man für die Ausgabe aller Elemente benutzen kann.

Die `Stream`-Methoden aus den Schritten 2–4 (`map`, `sorted` und `filter`) sind hier sogenannte **nicht-terminale** Methoden: Das bedeutet, diese geben nach einem Bearbeitungsschritt auf einem Stream einen neuen Stream zurück, auf dem weitere Bearbeitungsschritte (Methoden) angewendet werden können. Abgeschlossen wird die Bearbeitung eines Streams immer mit einer **terminalen** Methode, welche keinen Stream mehr zurückgibt – wie im Beispiel `forEach`.

Was sind aber nun eigentlich Streams?

Ein Stream realisiert eine sogenannte **Pipeline**. Eine Pipeline kann man sich wie ein Fließband für Daten-Objekte vorstellen: Am Anfang kommen Daten-Objekte aus einer **Quelle**, auf ihnen werden nacheinander verschiedene Operationen durchgeführt und am Ende ein Ergebnis ausgegeben. Dabei werden die Objekte in der Pipeline nicht zwingend sequentiell durchlaufen (wie bei einem Iterator), sondern können teilweise auch **parallel** verarbeitet werden (was Streams sehr effizient macht).

Es gibt verschiedene mögliche Quellen für Streams. Dazu zählen Arrays (siehe obiges Beispiel), Textdateien und Sammlungen. Insbesondere lässt sich jede Sammlung mit der Default-Methode `stream()` der Schnittstelle `Collection<T>` in einen Stream umwandeln. Damit können wir auch eine Container-Klasse zu einem Stream machen. Dazu reichen wir mit einer neuen Container-Methode `stream()` den Stream der internen Sammlung `allForms` nach außen durch:

```
public class FormContainer implements Iterable<Form> {  
  
    private ArrayList<Form> allForms;  
  
    ...  
  
    public Stream<Form> stream() {  
        return allForms.stream();  
    }  
}
```

Ein Stream wird repräsentiert durch den parametrisierten Datentyp `Stream<T>`. Das ist eine Schnittstelle, die verschiedene (terminale und nicht-terminale) Operationen auf Streams definiert. Streams gibt es seit Java 8 und werden wie folgt benutzt:

Definition 5.19: Streams

Streams werden über die `stream`-Methode von Collections oder andere Quellen erzeugt. Die Benutzung eines Streams sieht dabei wie folgt aus:

```
<Stream>  
    .<ListOfNonTerminalOperations>  
    .<TerminalOperation>
```

- Auf jedem Stream kann nur einmal gearbeitet werden.
- Jede **nicht-terminale** Stream-Methode liefert als Rückgabe einen neuen Stream (bei Abarbeitung der Pipeline wird jede Operation also auf einem neuen Stream ausgeführt). Dadurch lassen sich beliebig viele nicht-terminale Methoden hintereinander getrennt durch einen Punkt aufrufen.
- Jede **terminale** Methode *verbraucht* den Stream – es wird **kein** neuer Stream zurückgegeben. Terminale Operationen werden daher immer als letzte von mehreren Stream-Operationen aufgerufen.

Wir wollen einige Methodenköpfe von Stream-Methoden exemplarisch genauer betrachten.

Beispiel 5.20: Einige nicht-terminale Methoden

- `Stream<T> filter(Predicate<? super T> predicate):`
Behält nur jene Elemente im Stream, die eine bestimmte Eigenschaft erfüllen. Die Eigenschaft wird durch die Schnittstellen-Methode eines `Predicate`-Objekts überprüft (dieses kann als Lambda-Ausdruck übergeben werden). Die Methode muss für Obertypen des Stream-Parameters `T` definiert sein.
- `Stream<R> map(Function<? super T, ? extends R> mapper):`
Wandelt die `T`-Elemente in `R`-Elemente um. Die Umwandlung erfolgt durch die Schnittstellen-Methode eines `Function`-Objekts (dieses kann als Lambda-Ausdruck übergeben werden). Die Methode muss einen Obertyp von `T` in einen Untertyp von `R` umwandeln.
- `Stream<T> sorted():`
Sortiert die Stream-Elemente nach ihrer natürlichen Ordnung gemäß der `Comparable`-Schnittstelle (Laufzeitfehler, falls `T` diese nicht implementiert).
- `Stream<T> sorted(Comparator<? super T> comparator):`
Sortiert die Stream-Elemente. Die Vergleichsregel für die Sortierung wird durch die Schnittstellen-Methode eines `Comparator`-Objekts vorgegeben (dieses kann als Lambda-Ausdruck übergeben werden).
- `static <T> Stream<T> generate(Supplier<? extends T> s):`
Erzeugt und liefert einen unendlichen Stream, wobei jedes Element durch die Schnittstellen-Methode des übergebenen `Supplier`-Objekts `s` generiert wird (`s` kann als Lambda-Ausdruck übergeben werden). Die Methode muss für Untertypen des Stream-Parameters `T` definiert sein.
- `static <T> Stream<T> iterate(T seed, UnaryOperator<T> f):`
Erzeugt und liefert einen unendlichen Stream, wobei jedes Element durch die Schnittstellen-Methode des übergebenen `UnaryOperator`-Objekts `f` generiert wird (`f` kann als Lambda-Ausdruck übergeben werden). Dazu wird mit `f` ein neues Element immer aus dem vorherigen Element erzeugt, beginnend mit `seed`.

Hierbei handelt es sich bei `generate` und `iterate` um statische Methoden. Recherchieren Sie selbstständig, was `<T>` hinter `static` bedeutet.

Beispiel 5.21: Einige terminale Methoden

- `void forEach(Consumer<? super T> action):`
Führt eine Operation auf allen Stream-Elementen aus. Die Operation wird durch die Schnittstellen-Methode eines `Consumer`-Objekts definiert (dieses kann als Lambda-Ausdruck übergeben werden).
- `long count():`
Gibt die Anzahl der Stream-Elemente zurück.

- `Optional<T> reduce(BinaryOperator<T> accumulator):`

Iteriert eine binäre Operation über alle Stream-Elemente, um diese zu aggregieren. Die Operation wird durch die Schnittstellen-Methode eines `BinaryOperator`-Objekts definiert (dieses kann als Lambda-Ausdruck übergeben werden). Die Operation muss assoziativ sein. Sie wird zuerst auf die ersten beiden Elemente angewendet, dann auf das sich daraus ergebende Zwischenergebnis und das 3. Element, und so weiter auf das jeweilige neue Zwischenergebnis und das jeweils nächste Element. Das Ergebnis wird als Inhalt eines `Optional`-Objekts zurückgegeben. Ist der Stream leer, so ist auch das `Optional`-Objekt leer.

Beispiel: Durch Übergabe des Lambda-Ausdruck $(n, m) \rightarrow n + m$ wird die Summe aller Elemente eines Zahlenstreams berechnet.

Als weiteres Beispiel wollen wir einen Stream mit 100 zufälligen Dezimalzahlen erzeugen und ausgeben:

```
Stream.generate(() -> {return Math.random();})
    .limit(100)
    .forEach(System.out::println);
```

Dazu benutzen wir zuerst die `generate`-Methode. Diese erwartet ein `Supplier`-Objekt als Eingabeparameter. `Supplier<T>` ist eine funktionale Schnittstelle mit genau einer abstrakten Methode, nämlich:

```
T get()
```

Die `generate`-Methode erzeugt einen unendlichen Stream durch wiederholten Aufruf der `get`-Methode des übergebenen `Supplier`-Objekts. Mit dem Lambda-Ausdruck

```
() -> {return Math.random();}
```

definieren wir die Implementierung von `get`. Links vom Pfeil stehen mit `()` die Eingabeparameter von `get` (es gibt keine). Rechts vom Pfeil steht mit `return Math.random();` der Rumpf von `get`. Die Methode `get` erzeugt also bei jedem Aufruf eine zufällige Dezimalzahl (was ist also hier der Parameter `T`?).

Den so durch `generate` erzeugten Stream begrenzen wir mit `limit` auf 100 Zahlen und geben diese danach mit `forEach` aus.

Als weiteres Beispiel wollen wir die Summe der ersten 358 ungeraden Zahlen berechnen:

```
Optional<Integer> o =
    Stream.iterate(1, (Integer i) -> {return i + 2;})
        .limit(358)
        .reduce((Integer n, Integer m) -> {return n + m;});
if (o.isPresent())
    System.out.println(o.get());
else
    System.out.println("Stream ist leer");
```

Dazu benutzen wir zuerst die `iterate`-Methode. Diese erwartet als Eingabeparameter einen Anfangswert `seed` und ein `UnaryOperator`-Objekt. `UnaryOperator<T>` ist eine funktionale Schnittstelle mit genau einer abstrakten Methode, nämlich:

```
T apply(T o)
```

Die `iterate`-Methode erzeugt einen unendlichen Stream durch wiederholten Aufruf der `apply`-Methode des übergebenen `UnaryOperator`-Objekts. Hierbei wird an `apply` immer das zuvor erzeugte Objekt übergeben, beginnend mit `seed`. Mit dem Lambda-Ausdruck

```
(Integer i) -> {return i + 2;}
```

definieren wir die Implementierung von `apply`. Links vom Pfeil stehen mit `(Integer i)` die Eingabeparameter von `apply`. Rechts vom Pfeil steht mit `return i + 2;` der Rumpf von `apply`. Die Methode `apply` erzeugt also beim ersten Aufruf aus dem Anfangswert 1 den Wert 3, beim nächsten Aufruf aus 3 den Wert 5, und so weiter. Es wird also ein Stream aller ungeraden Zahlen erzeugt.

Den so durch `iterate` erzeugten Stream begrenzen wir mit `limit` auf 358 Zahlen.

Zuletzt benutzen wir die `reduce`-Methode. Diese erwartet als Eingabeparameter ein Objekt vom Typ `BinaryOperator<T>`. `BinaryOperator<T>` ist eine funktionale Schnittstelle mit genau einer abstrakten Methode, nämlich:

```
T apply(T o, T u)
```

Die `reduce`-Methode aggregiert die Elemente eines Streams durch wiederholten Aufruf der `apply`-Methode des übergebenen `BinaryOperator`-Objekts und liefert das Ergebnis. Hierbei wird an `apply` immer das nächste Stream-Element und das Ergebnis des letzten Aufrufs von `apply` übergeben, beginnend mit den ersten beiden Elementen. Mit dem Lambda-Ausdruck

```
(Integer n, Integer m) -> {return n + m;}
```

definieren wir die Implementierung von `apply`. Links vom Pfeil stehen mit

```
(Integer n, Integer m)
```

die Eingabeparameter von `apply`. Rechts vom Pfeil steht mit

```
{return n + m;}
```

der Rumpf von `apply`. Die Methode `apply` berechnet also beim ersten Aufruf die Summe der ersten beiden Elemente (das entspricht $1 + 3 = 4$), beim nächsten Aufruf die Summe aus dem Ergebnis des letzten Aufrufs und dem dritten Element (das entspricht $4 + 5 = 9$), und so weiter. Es wird also die Summe aller Elemente des Streams berechnet.

Der Rückgabotyp von `reduce` ist `Optional<T>`: Ist der Stream leer, so kann die `reduce`-Methode keinen Wert berechnen. Eine Möglichkeit wäre, diesen Fall durch die Rückgabe von `null` zu repräsentieren. Es liegt dann am Aufrufer, dies zu überprüfen und entsprechend zu reagieren. Das macht Code jedoch fehleranfällig, denn vergisst der Aufrufer die Überprüfung und ruft Methoden für die zurückgegebene Referenz auf, so führt das im Fall eines leeren Streams zu einem Programmabbruch zur Laufzeit. Das wird vermieden, indem ein `Optional`-Objekt zurückgegeben wird. Dies ist ein

Behälter (ähnlich einem Hüllklassen-Objekt), der entweder eine `T`-Referenz enthält, die nicht `null` ist, oder leer ist.

Mit der `isPresent`-Methode können wir zuerst testen, ob der Behälter leer ist. Ist dies nicht der Fall, können wir mit der `get`-Methode dann auf dessen Inhalt zugreifen.

Definition 5.22: Die Klasse `Optional<T>`

Ein `Optional<T>`-Objekt ist ein Behälter (ähnlich einem Hüllklassen-Objekt), der entweder eine `T`-Referenz enthält, die nicht `null` ist, oder leer ist.

Es gibt statische Methoden zur Erzeugung von nicht-leeren (`of`-Methode) und leeren (`empty`-Methode) `Optional`-Objekten.

Mit der `isPresent`-Methode kann getestet werden, ob der Behälter leer ist, und mit der `get`-Methode kann auf dessen Inhalt zugegriffen werden.

Weitere schöne Beispiele, Anwendungen und Einsatzmöglichkeiten für Streams finden Sie im Screenshot *Not-just-Chatting-Streams-in-Java*.